STAFF REVIEW · H446 317 source models

Separate from the eight-page pupil workbook. This copy is in a public repository and is NOT access controlled. Check technical correctness before use.

Printed question mapping: Booklet Q1 = original online Q1; Booklet Q2 = original online Q2; Booklet Q3 = original online Q3

Do Now — source answers

1. Requirement that must be satisfied for correctness.

2. Property maintained as iterations progress.

3. Additional memory remains constant with input size.

4. Three distinct values; three consecutive runs.

I Do — source worked steps

1. Explain the worked method — The loop starts at index1 and compares each item with its predecessor.

We Do — guided CHECK

1. Explain the worked method
The comparison counts new groups, not every repeated value.

You Do — source mark schemes

1. Mark scheme [8]: test for an empty array [1] and return 0 [1]; initialise count to 1 [1]; loop from index 1 to the last valid index [1]; compare adjacent values with != [2]; increment on change [1]; return count [1]. An equivalent clear pseudocode solution earns the same marks.
CODE:
```
function countDistinct(values)
if values.length == 0 then
return 0
endif
count = 1
for i = 1 to values.length - 1
if values[i] != values[i-1] then
count = count + 1
endif
next i
return count
endfunction
```

2. Start1 [1]; change to5→2 [2]; to6→3 [2]; to9→4 [2]; O(n) time [1] and O(1) additional space [1].

3. Indicative content: adjacent counting requires sorted input; after sorting, counting is O(n) time with O(1) extra counting state; comparison sorting often costs O(n log n), whereas a hash set can count distinct values in expected O(n) time with O(n) extra space and possible collision overhead. Practice marking (authored guidance, not an OCR mark scheme): 7–9 = accurate, sustained evaluation of both options, applied to this scenario with a supported judgement; 4–6 = relevant

comparison with some development and a partly justified decision; 1–3 = basic statements, limited context or unsupported decision; 0 = no creditworthy response. Use best-fit and credit equivalent valid arguments.
Model evaluation: The adjacent-comparison algorithm is excellent when the million values are already sorted: each value is inspected once, so time is O(n) and only a counter is needed. On unsorted data, identical values may be separated and it produces the wrong count unless the values are first sorted. Comparison sorting typically adds O(n log n) time, with space depending on the sorting method. A hash set processes an unsorted stream in expected O(n) time by recording each distinct value, but requires up to O(n) additional memory and performance depends on hashing and collisions. With sufficient memory and a one-off unsorted input, I would choose a hash set; for an already sorted array or strict memory constraints, adjacent counting is preferable.

Exit Ticket — source answer

1. False. Separated duplicate values can be counted repeatedly.